EnergyShield AI

Product & Technical Implementation Document — React + Supabase Architecture

Prepared for: EnergyShield AI Platform Engineering
July 2026

1. Purpose of This Document

This document defines the **end-to-end implementation architecture** for EnergyShield AI — an enterprise platform consisting of a public landing page and five connected operational dashboards (Government, Procurement, Shipping, Refinery, Decision Intelligence).

It translates the individual feature specifications already written for each screen into:

- One consolidated **project structure** (React + TypeScript + Supabase)
- A single **Supabase data model** shared by all five dashboards
- A defined **frontend ↔ backend communication pattern**
- The **cross-dashboard workflow** that connects the five modules into one pipeline
- A **phased build roadmap** so the system can be implemented incrementally without rework

This is a build blueprint, not a design mockup — every feature listed in the five dashboard specs is mapped to a concrete table, service, hook, and component below.

---

2. System Overview

EnergyShield AI models one continuous pipeline:

```
Global Risk Signals
      ↓
GOVERNMENT DASHBOARD        (national risk monitoring, approval authority)
      ↓  approved recommendation
PROCUREMENT DASHBOARD       (supplier selection, purchase orders)
      ↓  approved purchase order
SHIPPING DASHBOARD          (tanker tracking, logistics, port ops)
      ↓  delivered shipment
REFINERY DASHBOARD          (inventory, production, maintenance)
      ↓  operational summary
DECISION INTELLIGENCE       (executive KPIs, strategy, reports)
```

Each dashboard **writes** its own domain data and **reads** a summarized feed from the dashboard before it. No dashboard reaches into another dashboard's private tables directly — all cross-dashboard communication happens through a small set of **shared workflow tables** (Section 6), kept in sync using Supabase **Postgres triggers / Edge Functions** and consumed on the frontend using Supabase **Realtime**.

This keeps the "responsibility boundaries" defined in each spec intact (e.g. Shipping must never rank suppliers, Refinery must never track tankers) while still giving a connected, live experience.

---

3. Technology Stack

Layer	Technology	Notes
Frontend framework	React 18 + TypeScript + Vite	Fast dev server, strict typing
Styling	TailwindCSS + shadcn/ui	Enterprise dark theme, design tokens shared across dashboards
Animation	Framer Motion	Timeline arrows, counters, hover states
Routing	React Router v6	Nested routes per dashboard
Charts	Recharts	Refinery, Decision Intelligence analytics
Maps	Leaflet / OpenStreetMap	Global Risk Map, Live Vessel Tracking
Data fetching / caching	TanStack Query (React Query)	Wraps every Supabase call, handles caching + refetch
Backend platform	Supabase	Postgres, Auth, Realtime, Storage, Edge Functions
Server logic	Supabase Edge Functions (Deno/TypeScript)	Cross-dashboard forwarding, AI-call proxying
AI providers (later phase)	OpenAI / Gemini API, News API, AIS API, Weather API	Called only from Edge Functions, never from the browser

---

4. Consolidated Project Structure

All five dashboards and the landing page live in **one React application**, sharing the same design system, auth session, and Supabase client — this avoids duplicating login, theming, and API logic five times.

```
energyshield-ai/
├── src/
│   ├── assets/
│   ├── components/
│   │   ├── common/               # Buttons, KPI cards, badges, status pills
│   │   ├── ui/                   # shadcn/ui primitives
│   │   ├── landing/               # Hero, Stats, Features, Workflow, TechStack, Footer
│   │   ├── government/            # GovernmentHeader, RiskCards, WorldMap, AIRiskPanel,
│   │   │                          # AlertList, RecommendationPanel, ReserveCard, NotificationCenter
│   │   ├── procurement/           # GovernmentRecommendationCard, SupplierCards, SupplierRanking,
│   │   │                          # PurchaseOrderTable, CompatibilityPanel, ContractTable,
│   │   │                          # ProcurementAssistant, NotificationPanel
│   │   ├── shipping/              # ShipmentQueue, ShipmentTracking, WorldMap, PortOperations,
│   │   │                          # WeatherPanel, RouteOptimizer, DeliveryTimeline,
│   │   │                          # RefineryImpactPanel, NotificationCenter
│   │   ├── refinery/               # RefineryStatus, InventoryCards, IncomingShipmentTable,
│   │   │                          # ProductionPlanning, CompatibilityPanel, MaintenancePlanner,
│   │   │                          # RiskAnalysis, OperationsAssistant, NotificationCenter
│   │   └── decision/               # ExecutiveSummary, DashboardHealth, ExecutiveInsights,
│   │                              # PerformanceCharts, EconomicImpact, ScenarioSimulator,
│   │                              # StrategyCenter, ReportsCenter, NotificationCenter
│   ├── pages/
│   │   ├── Landing/
│   │   ├── Login/
│   │   ├── GovernmentDashboard/
│   │   ├── ProcurementDashboard/
│   │   ├── ShippingDashboard/
│   │   ├── RefineryDashboard/
│   │   └── DecisionDashboard/
│   ├── layouts/
│   │   ├── PublicLayout.tsx        # Navbar + Footer for landing/login
│   │   └── DashboardLayout.tsx     # Sidebar + Topbar shared by all 5 dashboards
│   ├── routes/
│   │   ├── AppRoutes.tsx
│   │   └── ProtectedRoute.tsx      # Role-based route guard
│   ├── contexts/
│   │   ├── AuthContext.tsx         # Supabase session + user role
│   │   ├── ThemeContext.tsx        # Dark/light toggle
│   │   └── NotificationContext.tsx # Realtime notification stream
│   ├── hooks/
│   │   ├── useAuth.ts
│   │   ├── useRealtimeTable.ts     # Generic Supabase Realtime subscription hook
│   │   ├── useSupabaseQuery.ts     # Wraps React Query + supabase-js
│   │   └── useRole.ts
│   ├── services/                   # ONE file per domain — the only place that talks to Supabase
│   │   ├── authService.ts
│   │   ├── governmentService.ts
│   │   ├── procurementService.ts
│   │   ├── shippingService.ts
│   │   ├── refineryService.ts
│   │   ├── decisionService.ts
│   │   └── notificationService.ts
│   ├── lib/
│   │   ├── supabaseClient.ts       # Single Supabase client instance
│   │   └── queryClient.ts          # React Query client config
│   ├── types/
│   │   ├── government.ts
│   │   ├── procurement.ts
│   │   ├── shipping.ts
│   │   ├── refinery.ts
│   │   ├── decision.ts
│   │   └── database.ts             # Generated Supabase types (supabase gen types typescript)
│   ├── constants/
│   │   ├── roles.ts
│   │   └── statusEnums.ts
│   └── utils/
├── supabase/
│   ├── migrations/                 # SQL migration files (Section 6)
│   ├── functions/                  # Edge Functions (Section 8)
│   │   ├── forward-recommendation/
│   │   ├── forward-purchase-order/
│   │   ├── forward-shipment/
│   │   ├── generate-ai-insight/
│   │   └── generate-executive-summary/
│   └── config.toml
└── .env
```

This structure is a direct merge of the folder trees already specified in each of the five dashboard documents — nothing is renamed, they are simply placed as siblings under one `components/` and `pages/` tree instead of five separate projects.

---

5. Authentication & Role-Based Access Control

5.1 Roles

Role key	Dashboard	Typical user
`government`	Government Dashboard	MoPNG officer
`procurement`	Procurement Dashboard	IOCL / BPCL / HPCL / Reliance / Nayara procurement officer
`shipping`	Shipping Dashboard	Port authority, logistics manager
`refinery`	Refinery Dashboard	Refinery operations / plant head
`executive`	Decision Intelligence Dashboard	Secretary MoPNG, NITI Aayog, CEOs
`admin`	All	Platform administrator

5.2 Implementation

- **Supabase Auth** (email/password, extendable to SSO) issues the session.
- A `profiles` table (1:1 with `auth.users`) stores `full_name`, `organization`, and `role`.
- Every protected table uses **Row Level Security (RLS)** so a procurement user can never read government-only rows, and vice versa — this is what technically enforces the “this dashboard should never do X” rules from each spec, not just UI hiding.
- `ProtectedRoute.tsx` on the frontend checks `role` from `AuthContext` before rendering a dashboard route; RLS is the real enforcement layer, the route guard is just UX.
- Login button on the landing page routes to `/login` only (per spec) — no backend wiring yet in this phase.

```
-- profiles table
create table profiles (
  id uuid references auth.users primary key,
  full_name text,
  organization text,
  role text check (role in ('government','procurement','shipping','refinery','executive','admin')),
  created_at timestamptz default now()
);

alter table profiles enable row level security;
create policy "Users can read own profile" on profiles
  for select using (auth.uid() = id);
```

---

6. Supabase Database Schema

The schema is grouped by dashboard, plus a small set of **shared workflow tables** that carry data across dashboard boundaries. All tables use `created_at` / `updated_at`, and every “status” field is a constrained enum so the frontend can render consistent progress pills.

6.1 Government Dashboard tables

```sql
create table national_risk_score (
  id uuid primary key default gen_random_uuid(),
  score numeric,
  risk_level text check (risk_level in ('low','medium','high','critical')),
  recorded_at timestamptz default now()
);

create table supplier_countries (
  id uuid primary key default gen_random_uuid(),
  name text,
  risk_level text check (risk_level in ('green','yellow','red')),
  current_imports numeric,
  active_events text,
  expected_impact text
);

create table ai_risk_insights (
  id uuid primary key default gen_random_uuid(),
  insight_text text,
  confidence_score numeric,
  data_sources text[],
  expected_impact text,
  created_at timestamptz default now()
);

create table government_alerts (
  id uuid primary key default gen_random_uuid(),
  title text,
  priority text check (priority in ('low','medium','high','critical')),
  affected_region text,
  recommended_action text,
  status text check (status in ('open','monitoring','resolved')) default 'open',
  created_at timestamptz default now()
);

create table strategic_petroleum_reserve (
  id uuid primary key default gen_random_uuid(),
  reserve_level numeric,
  remaining_days integer,
  consumption_rate numeric,
  ai_recommendation text,
  recorded_at timestamptz default now()
);

create table government_recommendations (
  id uuid primary key default gen_random_uuid(),
  title text,                       -- e.g. "Increase procurement from Russia"
  reason text,
  priority text check (priority in ('low','medium','high','critical')),
  confidence numeric,
  business_impact text,
  status text check (status in ('pending','approved','rejected','forwarded'))
        default 'pending',
  approved_by uuid references profiles(id),
  created_at timestamptz default now()
);
```

6.2 Procurement Dashboard tables

```sql
create table suppliers (
  id uuid primary key default gen_random_uuid(),
  country text,
  price_per_barrel numeric,
  supply_capacity numeric,
  delivery_time_days integer,
  geopolitical_risk text check (geopolitical_risk in ('green','yellow','red')),
  reliability_score numeric,
  contract_status text
);

create table supplier_ranking (
  id uuid primary key default gen_random_uuid(),
  supplier_id uuid references suppliers(id),
  rank integer,
  reasoning text,
  confidence numeric,
  business_impact text,
  ranked_at timestamptz default now()
);

create table purchase_orders (
  id uuid primary key default gen_random_uuid(),
  po_number text unique,
  recommendation_id uuid references government_recommendations(id), -- traceability
  supplier_id uuid references suppliers(id),
  quantity numeric,
  destination_refinery text,
  expected_delivery date,
  status text check (status in
      ('pending','approved','completed','delayed')) default 'pending',
  created_at timestamptz default now()
);

create table crude_compatibility (
  id uuid primary key default gen_random_uuid(),
  refinery_name text,
  crude_type text,
  compatibility_score numeric,
  expected_yield numeric,
  recommendation text
);

create table contracts (
  id uuid primary key default gen_random_uuid(),
  supplier_id uuid references suppliers(id),
  start_date date,
  end_date date,
  renewal_status text,
  ai_suggestion text
);
```

6.3 Shipping Dashboard tables

```sql
create table shipments (
  id uuid primary key default gen_random_uuid(),
  purchase_order_id uuid references purchase_orders(id),  -- traceability
  vessel_name text,
  supplier_country text,
  destination_port text,
  destination_refinery text,
  quantity numeric,
  current_lat numeric,
  current_lng numeric,
  current_speed numeric,
  eta timestamptz,
  status text check (status in
      ('preparing','departed','in_transit','delayed','arrived')) default 'preparing',
  created_at timestamptz default now()
);

create table ports (
  id uuid primary key default gen_random_uuid(),
  name text,                      -- Paradip, Sikka, Vadinar, Mundra, Vizag, Mumbai, Ennore
  waiting_ships integer,
  available_berths integer,
  avg_waiting_time numeric,
  congestion_level text check (congestion_level in ('green','yellow','red'))
);

create table weather_alerts (
  id uuid primary key default gen_random_uuid(),
  alert_type text,                -- cyclone, high waves, storm, fog
  affected_area text,
  expected_delay_days numeric,
  confidence_score numeric,
  ai_recommendation text
);

create table route_recommendations (
  id uuid primary key default gen_random_uuid(),
  shipment_id uuid references shipments(id),
  recommended_route text,
  time_saved_hours numeric,
  fuel_savings numeric,
  risk_reduction numeric,
  status text check (status in ('pending','approved','ignored')) default 'pending'
);
```

6.4 Refinery Dashboard tables

```sql
create table refinery_inventory (
  id uuid primary key default gen_random_uuid(),
  refinery_name text,
  current_inventory_barrels numeric,
  daily_consumption numeric,
  min_safety_stock numeric,
  remaining_days numeric,
  recorded_at timestamptz default now()
);

create table incoming_shipments (
  id uuid primary key default gen_random_uuid(),
  shipment_id uuid references shipments(id),  -- traceability into Shipping module
  refinery_name text,
  expected_arrival timestamptz,
  quantity numeric,
  status text check (status in ('pending','delivered','delayed')) default 'pending'
);

create table production_plan (
  id uuid primary key default gen_random_uuid(),
  refinery_name text,
  petrol_pct numeric,
  diesel_pct numeric,
  atf_pct numeric,
  lpg_pct numeric,
  lubricants_pct numeric,
  ai_recommendation text,
  reason text
);

create table maintenance_schedule (
  id uuid primary key default gen_random_uuid(),
  refinery_name text,
  unit_name text,
  status text,
  maintenance_due date,
  estimated_downtime_hours numeric,
  ai_recommendation text
);

create table production_risk (
  id uuid primary key default gen_random_uuid(),
  refinery_name text,
  inventory_remaining_days numeric,
  incoming_shipment_days numeric,
  risk_level text check (risk_level in ('low','medium','high','critical')),
  business_impact text
);
```

6.5 Decision Intelligence tables (read-only aggregation)

```sql
create table executive_kpis (
  id uuid primary key default gen_random_uuid(),
  energy_security_score numeric,
  national_risk_level text,
  supply_chain_health numeric,
  avg_delivery_performance numeric,
  refinery_utilization numeric,
  economic_impact numeric,
  recorded_at timestamptz default now()
);

create table dashboard_health (
  id uuid primary key default gen_random_uuid(),
  dashboard_name text,             -- government, procurement, shipping, refinery
  status text,
  health_score numeric,
  pending_actions integer,
  critical_issues integer
);

create table executive_insights (
  id uuid primary key default gen_random_uuid(),
  insight_text text,
  confidence_score numeric,
  business_impact text,
  affected_department text,
  created_at timestamptz default now()
);

create table economic_impact (
  id uuid primary key default gen_random_uuid(),
  estimated_loss numeric,
  logistics_cost_increase numeric,
  import_cost_increase numeric,
  fuel_price_impact numeric,
  inventory_holding_cost numeric,
  projected_savings numeric
);

create table scenario_simulations (
  id uuid primary key default gen_random_uuid(),
  scenario_name text,
  duration_days integer,
  estimated_shortage numeric,
  expected_price_increase numeric,
  affected_refineries text[],
  expected_loss numeric,
  recommended_actions text,
  run_at timestamptz default now()
);

create table strategic_recommendations (
  id uuid primary key default gen_random_uuid(),
  title text,
  priority text,
  confidence numeric,
  estimated_cost numeric,
  estimated_benefit numeric,
  long_term_impact text,
  status text check (status in ('pending','approved','archived')) default 'pending'
);
```

6.6 Shared workflow & notification tables

```sql
create table notifications (
  id uuid primary key default gen_random_uuid(),
  target_role text,                -- government | procurement | shipping | refinery | executive
  title text,
  message text,
  is_read boolean default false,
  created_at timestamptz default now()
);
```

Note the traceability columns above (`recommendation_id`, `purchase_order_id`, `shipment_id`, `incoming_shipments.shipment_id`) — these foreign keys are what technically implement the “forwarded to next dashboard” arrows drawn in every spec.

---

7. Frontend ↔ Backend Communication Pattern

All Supabase access is centralized so that no component ever imports the Supabase client directly.

```
Component
   │  calls a hook
   ▼
useSupabaseQuery() / useRealtimeTable()   (hooks/)
   │  calls a typed function
   ▼
governmentService.ts / procurementService.ts / ...   (services/)
   │  uses
   ▼
supabaseClient.ts   (lib/)
   │
   ▼
Supabase (Postgres + Auth + Realtime + Storage)
```

Example — services/governmentService.ts

```ts
import { supabase } from "@/lib/supabaseClient";
import type { GovernmentRecommendation } from "@/types/government";

export async function getRecommendations() {
  const { data, error } = await supabase
    .from("government_recommendations")
    .select("*")
    .order("created_at", { ascending: false });
  if (error) throw error;
  return data as GovernmentRecommendation[];
}

export async function approveRecommendation(id: string, userId: string) {
  const { error } = await supabase
    .from("government_recommendations")
    .update({ status: "approved", approved_by: userId })
    .eq("id", id);
  if (error) throw error;
}
```

Example — hooks/useSupabaseQuery.ts (React Query wrapper)

```ts
import { useQuery } from "@tanstack/react-query";
import { getRecommendations } from "@/services/governmentService";

export function useGovernmentRecommendations() {
  return useQuery({
    queryKey: ["government_recommendations"],
    queryFn: getRecommendations,
  });
}
```

Example — hooks/useRealtimeTable.ts (generic live subscription)

```ts
import { useEffect } from "react";
import { useQueryClient } from "@tanstack/react-query";
import { supabase } from "@/lib/supabaseClient";

export function useRealtimeTable(table: string, queryKey: string[]) {
  const queryClient = useQueryClient();

  useEffect(() => {
    const channel = supabase
      .channel(`realtime:${table}`)
      .on("postgres_changes", { event: "*", schema: "public", table }, () => {
        queryClient.invalidateQueries({ queryKey });
      })
      .subscribe();

    return () => { supabase.removeChannel(channel); };
  }, [table, queryClient, queryKey]);
}
```

Every dashboard page combines both: a React Query hook for the initial fetch, and `useRealtimeTable` for the same table so cards refresh live without a manual reload — this is how “AI automatically loads latest intelligence” and similar realtime requirements in the specs are satisfied.

---

8. Cross-Dashboard Workflow (Edge Functions)

Rather than letting the frontend write directly into the *next* dashboard's tables (which would blur the responsibility boundaries each spec insists on), each handoff is done through a small **Supabase Edge Function**. The current dashboard calls its own function; the function performs the insert into the next module and creates the relevant notification — this keeps validation and business rules server-side.

Edge Function	Triggered from	Action
`forward-recommendation`	Government → "Approve" button	Marks `government_recommendations.status = forwarded`, inserts row visible to Procurement, inserts a `notifications` row with `target_role = procurement`
`forward-purchase-order`	Procurement → "Track Shipment"	Marks PO `approved`, inserts row into `shipments`, notifies `target_role = shipping`
`forward-shipment`	Shipping → "Notify Refinery"	Inserts into `incoming_shipments`, notifies `target_role = refinery`
`forward-refinery-summary`	Refinery → nightly / on demand	Aggregates inventory + production into `executive_kpis` / `dashboard_health`, notifies `target_role = executive`
`generate-ai-insight`	Any dashboard's AI panel	Proxies a prompt to OpenAI/Gemini (kept server-side so API keys never reach the browser), stores result in the relevant `*_insights` table

Example skeleton (`supabase/functions/forward-recommendation/index.ts`):

```
import { createClient } from "npm:@supabase/supabase-js@2";

Deno.serve(async (req) => {
  const { recommendationId } = await req.json();
  const supabase = createClient(
    Deno.env.get("SUPABASE_URL")!,
    Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
  );

  await supabase
    .from("government_recommendations")
    .update({ status: "forwarded" })
    .eq("id", recommendationId);

  await supabase.from("notifications").insert({
    target_role: "procurement",
    title: "New Government Recommendation",
    message: "A new recommendation has been forwarded for procurement review.",
  });

  return new Response(JSON.stringify({ success: true }), { status: 200 });
});
```

In the current phase (per the specs, "no backend logic yet") these functions can be **stubbed to only update status locally with mock data**; the moment real orchestration is needed, only the function body changes — the frontend call signature stays the same.

---

9. Realtime & Notification Architecture

- Every dashboard subscribes only to its **own** tables plus the shared `notifications` table filtered by its `target_role`.
- `NotificationContext.tsx` opens one Supabase Realtime channel per session on login, filtered server-side:

```
supabase
  .channel("notifications")
  .on("postgres_changes",
      { event: "INSERT", schema: "public", table: "notifications", filter: `target_role=eq.${role}` },
      (payload) => addNotification(payload.new))
  .subscribe();
```

- This single pattern implements the "Notification Center" section that appears, with dashboard-specific content, in all five specs.

---

10. Feature-to-Implementation Map

10.1 Landing Page

Feature	Implementation
Hero animated supply-chain illustration	SVG built with Framer Motion `path` animations (`components/landing/HeroIllustration.tsx`); no backend needed
Animated statistic counters	`components/landing/StatsSection.tsx` using a `useCountUp` hook, values from `constants/` for now
Feature cards, workflow timeline, tech logos	Static content components, Framer Motion scroll-triggered reveals
Dashboard preview cards	Link to `/login?redirect=/government` etc. — routing only, per spec
Login button	Routes to `/login` , no auth call yet

10.2 Government Dashboard

Feature	Table(s)	Component
National risk score, import status, reserve, alerts, predictions KPIs	`national_risk_score` , `strategic_petroleum_reserve` , `government_alerts`	`RiskCards`
Global risk map with hover detail	`supplier_countries`	`WorldMap`
AI risk intelligence panel	`ai_risk_insights`	`AIRiskPanel`
Active alerts w/ recommended action	`government_alerts`	`AlertList`
Strategic Petroleum Reserve panel + Release/Monitor/Report buttons	`strategic_petroleum_reserve`	`ReserveCard`
AI recommendations w/ Approve → confirmation → “Forwarded to Procurement”	`government_recommendations` + `forward-recommendation` function	`RecommendationPanel`
Notification Center (government-only)	`notifications` filtered `target_role=government`	`NotificationCenter`

10.3 Procurement Dashboard

Feature	Table(s)	Component
Government Recommendations inbox, Accept/Reject	`government_recommendations` (read), local status	`GovernmentRecommendationCard`
Supplier cards (price, capacity, delivery, risk, reliability)	`suppliers`	`SupplierCards`
AI supplier ranking with reasoning	`supplier_ranking`	`SupplierRanking`
Purchase order management + Track Shipment	`purchase_orders` + `forward-purchase-order` function	`PurchaseOrderTable`
Crude compatibility analysis	`crude_compatibility`	`CompatibilityPanel`
Contract management + renewal AI suggestion	`contracts`	`ContractTable`
Conversational AI assistant	`generate-ai-insight` function (mock responses in this phase)	`ProcurementAssistant`
Procurement-only notifications	`notifications` filtered `target_role=procurement`	`NotificationPanel`

10.4 Shipping Dashboard

Feature	Table(s)	Component
Shipment queue from Procurement, Dispatch/Cancel	`shipments`	`ShipmentQueue`
Active shipment tracking cards + progress bar	`shipments`	`ShipmentTracking`
Interactive live vessel map	`shipments` (lat/lng)	`WorldMap` (Leaflet)
Port operations (7 ports, congestion)	`ports`	`PortOperations`
Weather intelligence alerts	`weather_alerts`	`WeatherPanel`
AI route optimization, Approve/Ignore	`route_recommendations`	`RouteOptimizer`
Delivery timeline (departure → customs → refinery)	derived from `shipments.status`	`DeliveryTimeline`
Refinery impact analysis + Notify Refinery	`refinery_inventory` (read) + `forward-shipment` function	`RefineryImpactPanel`
Logistics-only notifications	`notifications` filtered `target_role=shipping`	`NotificationCenter`

10.5 Refinery Dashboard

Feature	Table(s)	Component
Refinery KPI cards (inventory, operating days, efficiency, health)	`refinery_inventory`	`RefineryStatus`
Crude inventory + trend chart, auto-updates on receipt	`refinery_inventory`	`InventoryCards`
Incoming shipments + Receive Shipment button	`incoming_shipments`	`IncomingShipmentTable`
Production planning (Petrol/Diesel/ATF/LPG/Lubricants) + AI adjustment	`production_plan`	`ProductionPlanning`
Crude compatibility per shipment	`crude_compatibility` (shared)	`CompatibilityPanel`
Maintenance planning + AI postpone/approve logic	`maintenance_schedule`	`MaintenancePlanner`
AI production risk analysis	`production_risk`	`RiskAnalysis`
Conversational operations assistant	`generate-ai-insight` function	`OperationsAssistant`
Refinery-only notifications	`notifications` filtered `target_role=refinery`	`NotificationCenter`

10.6 Decision Intelligence Dashboard (read-only)

Feature	Table(s)	Component
Executive KPI summary	`executive_kpis`	`ExecutiveSummary`
Dashboard health overview (5 cards, click-through)	`dashboard_health`	`DashboardHealth`
AI executive insights	`executive_insights`	`ExecutiveInsights`
Performance analytics charts	aggregated from `suppliers`, `shipments`, `ports`, `refinery_inventory`, `production_plan`	`PerformanceCharts`
Economic impact analysis	`economic_impact`	`EconomicImpact`
Scenario simulator + Run Simulation	`scenario_simulations`	`ScenarioSimulator`
Strategic recommendation center + Approve/Archive	`strategic_recommendations`	`StrategyCenter`
Reports center (Preview / Export PDF / Download)	Supabase Storage bucket `reports/`	`ReportsCenter`
Executive-only notifications	`notifications` filtered `target_role=executive`	`NotificationCenter`

---

11. Phased Implementation Roadmap

Building all six screens in parallel is not realistic — this order lets each phase demo end-to-end before the next begins.

Phase	Scope	Depends on
0	Project scaffold: Vite + TS + Tailwind + shadcn/ui, folder structure, Supabase project created, `.env` wired	—
1	Landing page (fully static/mock, no auth)	Phase 0
2	Supabase Auth + `profiles` + RBAC + `ProtectedRoute` + `/login` screen	Phase 0
3	Government Dashboard (all tables in §6.1) with mock-seeded data, realtime wired	Phase 2
4	Procurement Dashboard + `forward-recommendation` function, consuming Government output	Phase 3
5	Shipping Dashboard + `forward-purchase-order` function, consuming Procurement output	Phase 4
6	Refinery Dashboard + `forward-shipment` function, consuming Shipping output	Phase 5
7	Decision Intelligence Dashboard, aggregating all four prior dashboards (read-only)	Phase 6
8	Real AI integration: replace mock insight text with OpenAI/Gemini calls inside `generate-ai-insight`; wire News/Weather/AIS APIs into Government risk map, Shipping weather panel, and Refinery risk analysis	Phase 7
9	Hardening: RLS audit, Storage-based report export, notification polish, responsive QA across desktop/laptop/tablet/mobile	Phase 8

Each phase ships a working, demoable dashboard before the next one starts consuming its data — this mirrors the “input/output” boundaries already defined in the five specs.

---

12. Security & Non-Functional Notes

- **Row Level Security is mandatory, not optional** — it is the only real enforcement of “this dashboard should never see/do X” from each spec; UI-level route guarding alone can be bypassed.
- **Service-role key never ships to the browser** — it lives only inside Edge Function environment variables, used for the cross-dashboard writes described in Section 8.
- **AI provider keys** (OpenAI/Gemini/News/Weather/AIS) are called only from Edge Functions.
- **Supabase Storage** (`reports` bucket, private) holds generated PDF reports for the Decision Intelligence “Reports Center”; signed URLs are issued per download.
- Responsive breakpoints follow Tailwind defaults (`sm/md/lg/xl`) and every dashboard layout collapses the sidebar to a drawer below `lg` .
- All five dashboards share one `queryClient` and one `supabaseClient` instance — no duplicate connections.

---

13. Summary

This document consolidates the five dashboard specifications and the landing page brief into a single implementable system:

- **One React app**, five role-gated dashboard sections, one shared design system.
- **One Supabase project**, with per-module tables plus a small shared workflow layer (`notifications` , and traceability foreign keys) that carries data from Government → Procurement → Shipping → Refinery → Decision Intelligence exactly as each spec’s “Connections” section describes.
- **RLS + Edge Functions** as the real enforcement of dashboard boundaries, not just UI hiding.
- **React Query + Supabase Realtime** as the standard pattern for every data-driven component, satisfying every “auto-update” / “24x7 monitoring” requirement across the specs.
- A **nine-phase roadmap** that lets each dashboard be built, seeded with mock data, and demoed independently, then wired live to the one before it.